Assignment 1 – Summary

GitHub repository

GitHub username	ishan-monie
Repository name	advanced-ml-hw1
Link to the code	https://github.com/ishan-monie/advanced-ml-hw1

All of the code used for this assignment is in this repository.

How to run the code

Step 1. Download the repository (green *Code* button, then *Download ZIP*) or clone it, and install the Python packages (Python 3.12 was used):

```
pip install numpy matplotlib scipy torch "gymnasium[classic-control]" ale-py
```

Step 2. Shuttle problem. In the `q1_shuttle` folder:

```
python run_q1.py     # solves the problem 3 ways and saves the 3 plots in figures/
python test_q1.py    # the 32 edge-case and correctness checks (PASS/FAIL for each)
```

Step 3. Policy gradient, no baseline. In the `q3_policy_gradient` folder:

```
python run_cartpole.py   # CartPole: trains, plays 500 test games, saves graphs
python test_cartpole.py  # 24 checks
python plot_pong.py <3 result folders>   # Pong graphs (folders: see README)
python test_pong.py      # 29 checks
```

Step 4. Baselines. In the `q4_baselines` folder:

```
python run_cartpole_baselines.py   # CartPole, 3 methods x 10 seeds (20-30 minutes)
python test_cartpole_baselines.py  # 22 checks
python plot_pong_baselines.py      # Pong baseline graphs
python test_pong_baseline.py       # Pong baseline and convolutional checks
```

Pong training takes hours, so the finished Pong runs are included in the repository. The README lists the exact commands that made them. Runs use fixed random seeds, but a different number of computer threads can change a run (see the CartPole baseline results), so the CartPole baseline runs use 1 thread.

Question 2: Value and Policy Iteration (the shuttle problem)

Each period we choose whether to dispatch a shuttle (cost 100, takes up to 15 people) or wait (every person left waiting costs 2). Between 1 and 5 new people arrive each period. We solve this three ways. A lower cost is better in every graph.

Assumptions

The problem leaves some details open, so these are the choices we made:

- **What we track.** The only thing that matters when deciding is how many people are waiting at the start of a period (0 to 200). We do not track the time of day or what happened before.
- **Costs, not rewards.** Everything is a cost, so we look for the *lowest* total cost. The “value” in the graphs is the expected total cost, so lower is better.
- **Order of events in a period.** We first decide (wait or dispatch), then pay, then the new people arrive.
- **Dispatching.** The shuttle is always available. It takes up to 15 people, or everybody if fewer than 15 are waiting. We allow dispatching with an empty station, even though it is wasteful.
- **The waiting cost.** The cost of 2 per person is charged on the people *left* after the shuttle has gone (everyone, if we wait).
- **Full station.** The station holds at most 200 people. If more would arrive, the extra people are turned away (they are lost, they do not wait). We checked that this choice does not change the answer.
- **Arrivals.** Each period 1, 2, 3, 4 or 5 people arrive, each with chance 1/5, independent of the past.
- **Discounting.** A cost one period later counts 0.95 as much. The cost in the first period is not discounted.
- **The end of the 500 periods (enumeration).** After the last period nothing more is paid, so people still waiting at that point cost nothing extra.
- **The plan.** The best action depends only on the number of people waiting, and it is the same in every period (this is what value iteration and policy iteration find).
- **Stopping and ties.** Value iteration stops when no value changes by more than 1e-10. Policy iteration starts from “never dispatch” and keeps its current action when two actions cost exactly the same.

(a) Enumeration, 500 periods

We start at the end of the 500 periods, where nothing is left to pay, and work backwards, keeping the cheaper of the two actions at every step. The graph shows the best expected total cost at time 0 for each number of people waiting.

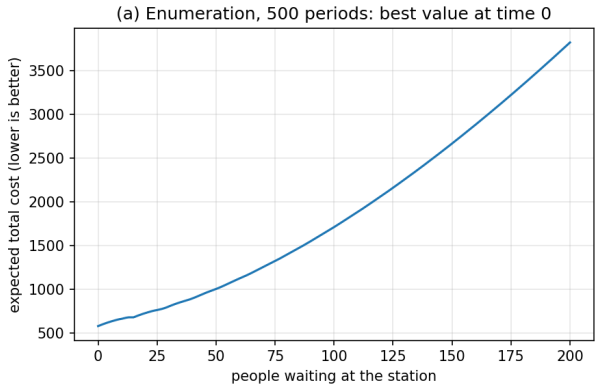


Figure 1. Enumeration: best expected cost at time 0 versus people waiting.

(b) Value iteration, no end date

The same step is repeated with no end date until the numbers stop changing (519 rounds). The curve is the same as in (a), which is expected: after 500 periods the far future is discounted to almost nothing, and the two differ by about 3 in a billion.

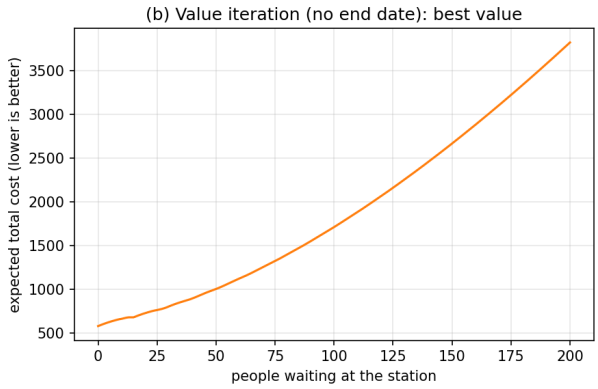


Figure 2. Value iteration: best expected cost versus people waiting.

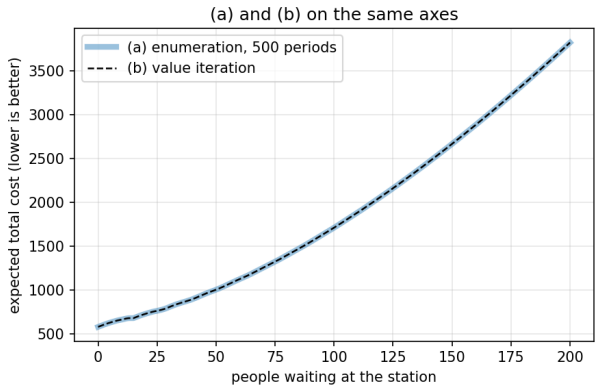


Figure 3. Figures 1 and 2 on the same axes. The two curves lie on top of each other.

(c) Policy iteration

Here we improve a plan instead of the costs. Starting from “never dispatch”, we measure how good the plan is, change it wherever another action is cheaper, and repeat. It settled after 6 rounds. The best plan is to wait while fewer than 13 people are waiting and to dispatch at 13 or more.

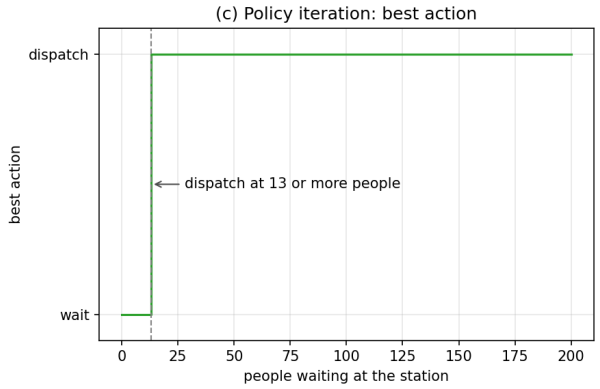


Figure 4. Policy iteration: best action versus people waiting.

Edge cases

The unusual situations we considered, and how the code handles each one (P1 to P17).

Built into the calculation

#	Unusual situation	How the code handles it
P1	Shuttle leaves with fewer than 15 people	It takes everybody; the number left is never below 0 (`max(people - 15, 0)`).
P2	Arrivals would push the station past 200	The station stops at 200 and the extra people are turned away (`min(..., 200)`).
P3	Chances of the next situation not adding up to 100%	Each possible arrival number gets exactly 1 / (number of options), so they always add up to 100%.
P4	Off-by-one: 0 to 200 is 201 situations, not 200	Every table is sized 200 + 1.
P5	Waiting should not remove anybody	When we wait, everyone stays; we pay 2 for each.
P6	Enumeration needs a starting point	After the last period the value is 0, then it works backwards from period 499 to 0.
P7	Value iteration stopping too early or never	It stops only when no value changes by more than 1e-10, not after a fixed number of rounds.
P8	Policy iteration looping on ties or rounding noise	It only changes an action if the new one is better by more than 1e-9; otherwise it keeps the current action.
P9	Value iteration flip-flopping on exact ties	Dispatch is chosen only if it is strictly cheaper; a tie means wait.
P10	Inaccurate evaluation of a plan	Policy iteration solves the exact equations instead of approximating.
P11	Dispatching an empty station	Allowed, with no special rule. It is just never the best action.

Safety checks added to stop bad inputs and runaway calculations

#	Unusual situation	How the code handles it
P12	Nonsense numbers: no arrivals, negative arrivals, sizes or costs, a station limit of 0, tables that do not match the problem	It stops at the start with a clear message saying what is wrong.
P13	A discount of 1 or more (or below 0)	Rejected. With a discount of 1 the “forever” problem has no finite answer.
P14	Value iteration never settles	A maximum number of rounds; if it is exceeded the program stops with an error instead of running forever.
P15	Policy iteration never settles	The same maximum for policy iteration.
P16	A strange horizon for enumeration (0, negative or a fraction of a period)	Rejected: it needs a whole number of periods, at least 1.
P17	A value turns into NaN or infinity	The calculation stops with an error, so bad numbers cannot spread quietly.

We did not guard against things like typing text instead of a number: that is careless typing, not part of this problem, and the assignment fixes all the numbers.

Tests

Each test says what we expect to happen, and the result column says what did happen. All 32 tests passed, meaning every result matched the expectation. Run them with `python test_q1.py`. The last column says which edge case (P1 to P17) a test covers.

A. Ordinary situations

Test (and what we expect)	Result (matched what we expected)	Edge case
A1. Empty station (0 people): the best action should be to wait, because a shuttle would cost 100 to carry nobody.	The best action was to wait.	P11
A2. Dispatching with fewer than 15 people (4): the shuttle should take everybody, and we should pay only the 100.	The cost was 100 and the next period had 1 to 5 people.	P1
A3. Does the 200-person limit matter? With a limit of 300 the best actions should not change.	The actions were the same up to 150 people, and the costs differed by only 9e-11.	P2
A4. The chances of the next situation should add up to 100% everywhere.	The largest error was 0, and a deliberately damaged table was caught by the same check.	P3
A5. A bigger shuttle should wait for more people before being sent.	Shuttles of size 5, 15 and 30 dispatched at 5, 13 and 17 people.	–

B. Edges of the station

Test (and what we expect)	Result (matched what we expected)	Edge case
B1. Exactly 15 people should empty the station, while 16 should leave one behind.	The costs were 100 and 102.	P1
B2. Full station (200): we should dispatch, and waiting should not push it above 200.	The best action was to dispatch, and waiting stayed at 200.	P2
B3. Nearly full (199 and 198): at 199 every arrival should be turned away, and at 198 only one should fit.	From 199 the station always went to 200. From 198 it went to 199 with 20% chance and to 200 with 80%.	P2
B4. Off-by-one: there should be 201 situations (0 to 200), not 200.	All tables are 201 wide and the graphs have 201 points.	P4

C. Arrivals

Test (and what we expect)	Result (matched what we expected)	Edge case
C1. Nobody ever arrives and 10 are waiting (worked by hand): waiting forever costs 20 / (1 - 0.95) = 400 and dispatching once costs 100, so the cheapest should be 100.	The program gave 100.000.	–
C2. Far more arrivals than the shuttle can carry (16 to 20 per period): the queue never empties, so each person left costs 2 / (1 - 0.95) = 40 forever. A dispatch (100) should pay off from 100 / 40, so from 3 people. (Our first guess was 15; working it through showed the guess was wrong, not the program.)	It dispatched for every number from 3 to 152.	–
C2b. Why does it stop at 152? Near the limit the queue hits 200 anyway, so a dispatch saves little, and the stopping point should move if the limit moves.	The last dispatch was at 152 with a limit of 200, and at 252 with a limit of 300, so it is an effect of the cap.	P2

D. Extreme prices

Test (and what we expect)	Result (matched what we expected)	Edge case
D1a. Free shuttle: we should dispatch whenever anyone is waiting.	It dispatched in all 200 situations with someone waiting.	–
D1b. Free waiting: we should never dispatch, and the total cost should be 0.	It never dispatched, and the cost was 0.	–
D2. Enormous shuttle cost (1,000,000): we should never dispatch, even with a full station.	It never dispatched.	–

E. Time and discounting

Test (and what we expect)	Result (matched what we expected)	Edge case
E1. Very short horizon (worked by hand): with 1 period left the cost should be 2 times the number of people, and with 2 periods left 3.9 times the people plus 5.7.	Both matched the program exactly.	P6
E2. Is 500 periods long enough? As the horizon grows from 100 to 200 to 500 periods, the answer should settle down.	The difference from the no-end-date answer was 4.0, then 0.024, then 0.000000003.	P6

F. Checks on each method

Test (and what we expect)	Result (matched what we expected)	Edge case
F1. Enumeration starting point: nothing should be left to pay after the last period, and more periods left should never cost less.	The value after period 500 is 0 everywhere, and costs only grew with more periods.	P6
F2. Value iteration should reach the same answer from different starting guesses.	Starting guesses of all 0, all 1000 and random numbers agreed to 4e-9.	P7
F3. Policy iteration should improve the plan every round, and when all actions cost the same it should not loop forever.	It took 6 rounds and the cost never went up. A problem where every action ties stopped after 1 round.	P8, P9

G. Overall checks

Test (and what we expect)	Result (matched what we expected)	Edge case
G1. All three methods should agree on the costs and on the plan.	The biggest gap was 3e-9, and the plan was identical.	all
G2. Policy iteration started from a bad plan (“always dispatch”) should still find the same best plan.	It did.	P8
G3. Simulating the station 3000 times should give an average cost that matches the calculated cost.	The simulation gave 575.8 (plus or minus 1.7) against a calculated 576.7.	–

H. The safety checks themselves

Test (and what we expect)	Result (matched what we expected)	Edge case
H1. Nonsense numbers (no arrivals, negative arrivals, sizes or costs, a limit of 0) should each be rejected with a clear message.	All 6 were rejected.	P12
H1b. Tables of the wrong size, or whose chances do not add to 100%, should be rejected.	Both were stopped.	P12
H2. A discount of 1.0, 1.5 or -0.1 should be rejected, while a discount of 0.0 should be allowed.	The first three were rejected and 0.0 was accepted.	P13
H3. Value iteration given a target it can never meet should stop with an error after the maximum number of rounds.	It stopped with an error (50 rounds were allowed).	P14
H4. Policy iteration allowed only 1 round (it needs 6) should stop with an error.	It stopped with an error.	P15
H5. Enumeration with 0, -3 or 2.5 periods should be rejected.	All three were rejected.	P16
H6. A NaN or infinite cost should make all three methods stop with an error.	All three methods stopped. (An infinite cost on an action nobody uses is rightly not an error.)	P17
H7. Waiting with 10 people: we should pay 20, and next period should have 11 to 15 people.	The cost was 20, with a 20% chance of each of 11 to 15 people.	P5
H8. Exact equation solving should agree with a slow repeat-until-settled calculation.	The largest gap was 9e-13.	P10

With 3 more months

Make the model more realistic. We assumed 1 to 5 people arrive with equal chance. With real arrival data we would check whether “dispatch at 13” still holds, and add rush hours, travel time, several shuttles and people who give up waiting. We would also see how far the answer moves if the costs of 100 and 2 are really 50% to 150% of what we assumed, and ask someone who runs a station whether our assumptions (such as when the waiting cost is charged) match reality.

Test the solution harder. We would generate hundreds of random shuttle problems and check that all three methods agree on every one, compare against a brute-force search over every plan on small versions, and simulate rival rules (“dispatch at 10”, “at 15”, “at 20”) for a long time to show with confidence ranges that 13 really is cheapest.

Check it a different way. We would let a learning method, like the policy gradient methods in the next questions, learn from simulated stations without being told the rules, and see whether it also finds about 13. We would also check that the program stays fast and accurate on a much bigger station.

Question 3: Policy Gradient, Part 1

We teach a small neural network to play a game by trial and error. It outputs a probability for each action and plays a whole game choosing actions at random from those probabilities. Afterwards, actions that were followed by lots of reward are made more likely, and actions followed by little reward are made less likely. This is the basic version, with no baseline (the next part adds one).

Assumptions

These are the choices we made for part (a), CartPole-v1:

- **One game is one episode.** CartPole ends when the pole tips too far, the cart leaves the track, or after 500 steps. Every step the pole stays up gives a reward of 1, so the best possible score is 500.
- **The network.** It sees 4 numbers (cart position, cart speed, pole angle, pole speed), has one hidden layer of 128 units, and outputs the probability of pushing left or right.
- **Actions are random.** Both during training and during the final 500 games, each push is chosen at random using the network's probabilities, rather than always taking the most likely one.
- **Credit for each push.** Each push is credited with the reward from that step onwards, discounted by 0.95 per step (as the assignment says), not with the whole game's total.
- **Learning.** One update after every game, using Adam with a learning rate of 0.01 (as the assignment says). There is no baseline and the credits are not rescaled, because this is the basic version.
- **Training length and luck.** We train for 1500 games. Results depend on the random starting point, so the main model uses seed 0, chosen in advance, and two more seeds are shown to see how much luck matters.
- **Final test.** After training the network stops learning. We play 500 new games, with a different random seed, and plot the scores.
- **Moving average.** The average of the last 100 games (fewer at the very start).
- **The 500-step limit** is treated like any other end of the game.
- **Bad numbers.** If an update ever produced NaN or infinity, it would be skipped instead of damaging the network. This never happened in our runs.

(a) CartPole-v1

We tried learning rates of 0.01, 0.003 and 0.001 on three seeds each. The averages of the last 100 training games were 285, 249 and 234, but they varied a lot between seeds (for example 500 against 163 at the same rate), so no rate was clearly better and we kept the assignment's 0.01.

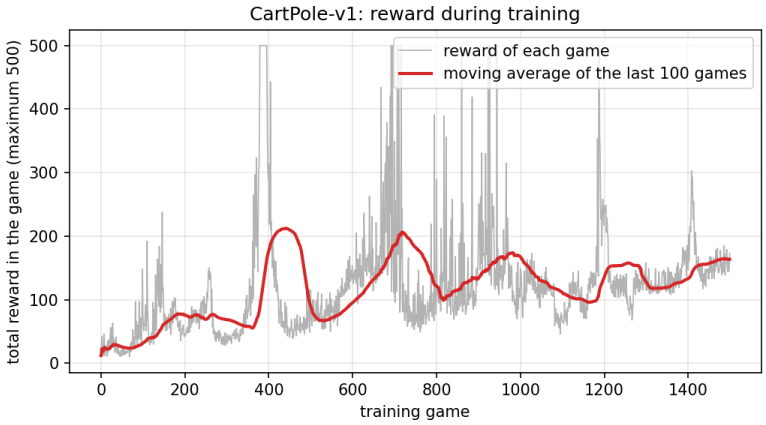


Figure 5. Reward of every training game (grey) and the moving average of the last 100 games (red).

The network clearly learns: early games last about 20 steps and later ones often reach 200 or more. But it is unsteady. The average rises, falls back, and rises again, and the main model finished at an average of about 164 over its last 100 training games. This noise is typical of the basic method and is what a baseline is meant to reduce.

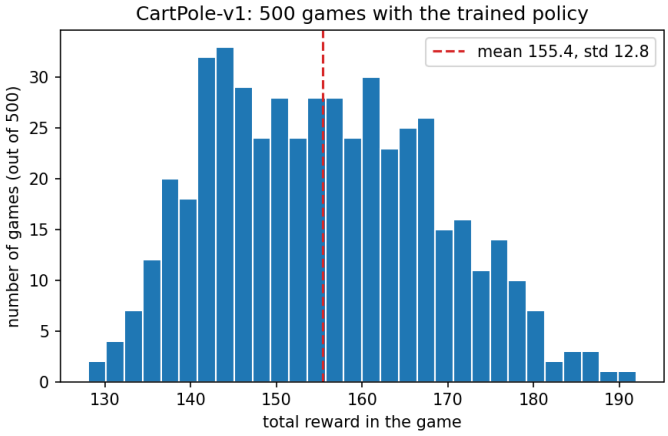


Figure 6. Scores of 500 games played by the trained main model (mean 155.4, standard deviation 12.8).

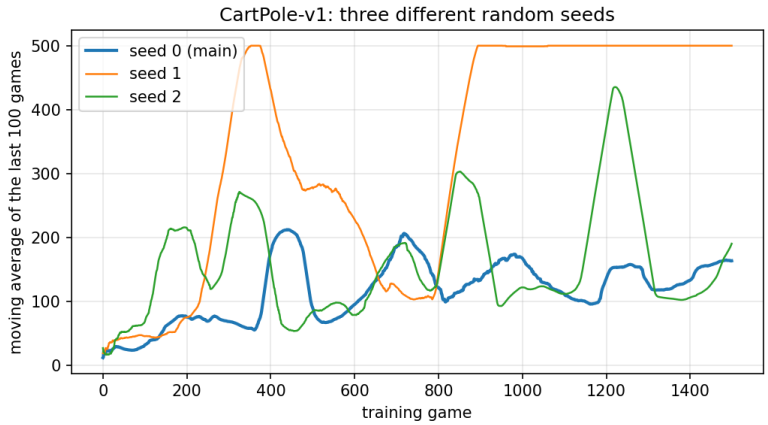


Figure 7. The same training with three different random seeds. Seed 1 reaches the 500 limit and stays there; seeds 0 and 2 do not.

On the 500 test games, the main model (seed 0) scored a mean of 155.4 with a standard deviation of 12.8. The other seeds scored a mean of 500.0 (seed 1, with no variation) and 277.5 (seed 2). Where the model ends up depends a lot on the seed, so a single run should not be over-interpreted.

CartPole: Edge cases

The unusual situations we considered for part (a), and how the code handles each one (R1 to R9).

#	Unusual situation	How the code handles it
R1	A reward crediting pushes that came before it	Credits are worked out backwards with a running total, so each push only gets the reward from its own step onwards.
R2	Bad settings: fewer than 1 game, a learning rate of 0 or below, a discount below 0 or of 1 or more	Rejected at the start with a clear message. With a discount of 1 or more the credits could grow without limit.
R3	A NaN or infinite number in the learning step	That update is skipped and counted, so the network cannot be damaged. A warning is printed if it ever happens (it did not in our runs).
R4	An empty or very short game	An empty game is skipped. A one-step game still gives a valid update.
R5	Taking the log of a probability that is 0, or huge numbers	Probabilities are built from the raw scores, so they are never exactly 0 and stay finite even for extreme networks or inputs.
R6	How a game ends	The pole falling and the 500-step limit both simply end the game. We use CartPole-v1, so a score can never be above 500.
R7	Test games changing the network by accident	Test games switch off learning information, and the network is not updated during them.
R8	Results that cannot be repeated	Random seeds are fixed, and the 500 test games use a different seed from training.
R9	The moving average at the start, before 100 games exist	It averages over however many games exist so far, instead of failing or counting empty games as 0.

CartPole: Tests

Each test says what we expect to happen, and the result column says what did happen. All 24 tests passed, meaning every result matched the expectation. Run them with `python test_cartpole.py`. The last column says which edge case (R1 to R9) a test covers.

A. Credits and game rules

Test (and what we expect)	Result (matched what we expected)	Edge case
A1. Worked by hand: rewards 1, 1, 1 with discount 0.5 should give credits 1.75, 1.5 and 1.0, and a reward should never credit earlier pushes.	The credits were 1.75, 1.5 and 1.0, and a lone reward of 5 credited only its own step.	R1
A2. A one-step game (the pole falls straight away) should still give a valid update.	The update worked and the network changed.	R4
A3. A discount of 0 should give each push only its own reward, 0.99 should work, and 1.0 or -0.1 should be rejected.	Discounts of 0 and 0.99 worked; 1.0 and -0.1 were rejected.	R2
A4. In the longest game (500 steps) no credit should exceed 1 / (1 - 0.95) = 20.	The largest credit was 20.000.	–
A5. We should be using CartPole-v1 with a 500-step limit (not v0, which stops at 200).	It is CartPole-v1, with a limit of 500.	R6
A6. Every step should give a reward of 1, so a game's score equals its length.	A sample game of 36 steps scored 36.	–

B. The network

Test (and what we expect)	Result (matched what we expected)	Edge case
B1. Before any training, both actions should be about equally likely.	The average chance of pushing left was 0.527.	–
B2. The two probabilities should add up to 100% for every state, with no NaN.	The largest error was about 1e-7.	R5
B3. An extreme network (weights multiplied by 10,000) and extreme inputs (millions) should still give finite log-probabilities.	They did.	R5

C. The training loop

Test (and what we expect)	Result (matched what we expected)	Edge case
C1. A NaN reward should make the update skip, leaving the network untouched.	The update was skipped and the weights were identical.	R3
C2. An empty game should be skipped instead of crashing.	It was skipped.	R4
C3. Bad settings (0 games, a learning rate of 0 or below) should be rejected with a message.	All three were rejected.	R2
C4. The same seed should give an identical training curve, and a different seed a different one.	Two runs with seed 3 were identical, and seed 4 differed.	R8
C5. Test games should never change the network.	The weights were identical before and after, and no learning information was kept.	R7
C6. A made-up problem with a known answer (one state where action 1 always pays 1 and action 0 pays 0): our learning signal should match the exact gradient of the expected reward.	The direction matched (1.0000) and the size was within 0.9%.	–
C6b. The previous check should really be able to fail: a learner with the sign flipped, or one that ignores the reward, should be caught.	Both were caught (a direction match of -1.00 instead of +1.00).	–
C7. In the same made-up problem, training should make the better action much more likely.	Its chance went from 0.47 to 1.00.	–

D. Does it behave as predicted?

Test (and what we expect)	Result (matched what we expected)	Edge case
D1. The trained network should be far better than an untrained one.	155.4 against 22.0 on average (a statistical test gave t = 176).	–
D2. With a tiny learning rate (1e-6) it should hardly learn and stay near random play (about 22).	The average of the last 100 games was 20.2.	–
D3. Taking away the randomness (always the most likely action) should do about as well or better.	155.9 against 155.4.	–
D4. The last 100 training games should score much higher than the first 100.	The average was 31.4 at the start and 163.5 at the end.	–

E. The 500 test games and the graphs

Test (and what we expect)	Result (matched what we expected)	Edge case
E1. There should be exactly 500 test games, each scoring between 1 and 500.	500 games, with scores from 128 to 192, a mean of 155.4 and a standard deviation of 12.8.	R6
E2. The moving average, worked by hand: [1, 2, 3, 4] with a window of 2 should give 1, 1.5, 2.5, 3.5. With a window of 100 the first value should be the first game, game 100 the mean of games 1 to 100, and game 101 should drop game 1.	All of these matched.	R9
E3. The saved model should reproduce exactly the 500 test scores behind the graphs in this report, and retraining from the same seed should repeat the start of the saved training curve.	The 500 test scores were identical, and so were the first 200 training games.	R8

(b) Pong-v5

The same method as for CartPole, now on the Atari game Pong. The network sees the screen, outputs the chance of moving the paddle up, and plays whole games. These are the choices we made for part (b):

- **The game.** We play the right-hand paddle against the computer, using ALE/Pong-v5. A game ends when one side reaches 21 points. Each point we win gives +1 and each point we lose gives −1, so a game scores between −21 and +21. Random play scores about −20.
- **The game is partly random.** In Pong-v5 each push is held for 4 screen frames, and with a 25% chance the game repeats our previous push instead of the new one. So even identical play gives different results, and scores vary from game to game.
- **What the network sees.** Each screen is cleaned with the assignment's `preprocess` function into an 80 x 80 grid of 0s and 1s (paddles and ball are 1). The network is shown the *difference* between this frame and the previous one, so it can tell which way things are moving. The first frame of a game has no previous frame, so its difference is all zeros. `preprocess` changes the picture it is given, so we always give it a copy.
- **The network.** 6,400 inputs, one hidden layer of 256 units, and one output: the chance of pushing up.
- **Actions.** Of the 6 actions in the game, we use only RIGHT (action 2, paddle up) and LEFT (action 3, paddle down), as the assignment says.
- **Actions are random.** Both in training and in the final 500 games, each push is chosen at random using the network's probability.
- **Credit for each push.** Discount 0.99, as the assignment says. The running total restarts at every point scored, so a push is only credited for the point it was part of (each point is treated as its own small game). This is a common choice for Pong, but it is our choice, not something the assignment states.
- **Learning.** Adam, with the loss averaged over 4 games per update (one update per game would be very noisy for a network this big). No baseline and no rescaling of the credits, because this is the basic version.
- **Learning rate.** The assignment gives 0.01 but allows a different value if training is more stable. We tried both 0.01 and 0.001. Our rule, set before seeing the results: the main model is the one with the higher average over its last 100 training games, and both are shown. A second run at 0.001 with a different random seed shows how much luck matters.
- **Training length.** A Pong game takes one to several seconds of computer time (longer as the player improves), so runs are given a fixed time limit instead of a fixed number of games. The two 0.001 runs had 6 hours on a GPU server (one card) and played 6,993 (main) and 6,534 games. The 0.01 run had 4 hours on a laptop (9,486 games). An earlier pair of 0.001 runs on a laptop was cut short when the laptop went to sleep (about 3,300

and 2,900 games) and was replaced by these longer runs.

- **Final test.** After training the network stops learning. We play 500 new games, with a different random seed, and plot the scores.
- **Bad numbers.** An update that would produce NaN or infinity is skipped, and skipped updates are counted in the saved settings.

(b) Pong-v5: graphs and results

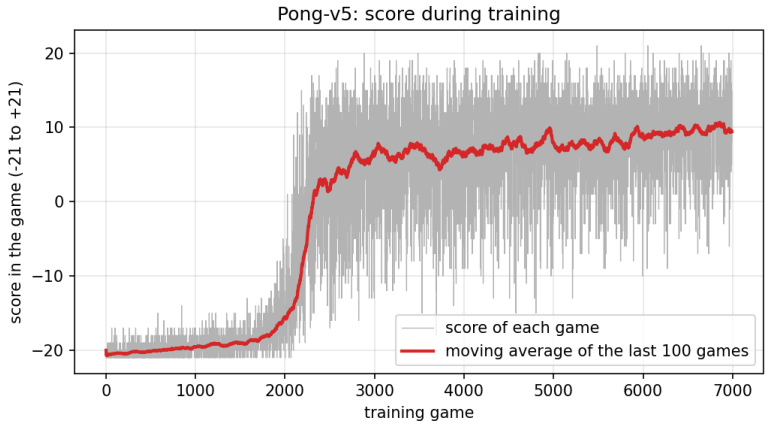


Figure 8. Score of every training game (grey) and the moving average of the last 100 games (red), main model (6,993 games).

The main model (learning rate 0.001, seed 0) stays near –21, where it loses almost every point, for about the first 2,000 games. Then it takes off, and it levels out at a score of about +8 to +10. Over its last 100 training games it averaged 9.4.

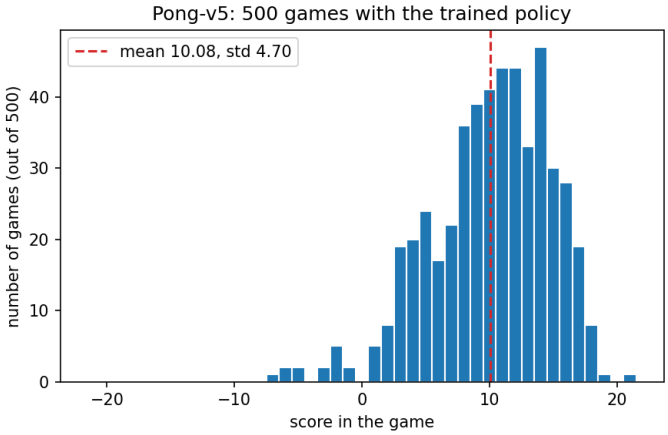


Figure 9. Scores of 500 games played by the trained main model (mean 10.08, standard deviation 4.70).

On the 500 test games the main model scored a mean of 10.08 with a standard deviation of 4.70, with scores from -7 to 21.

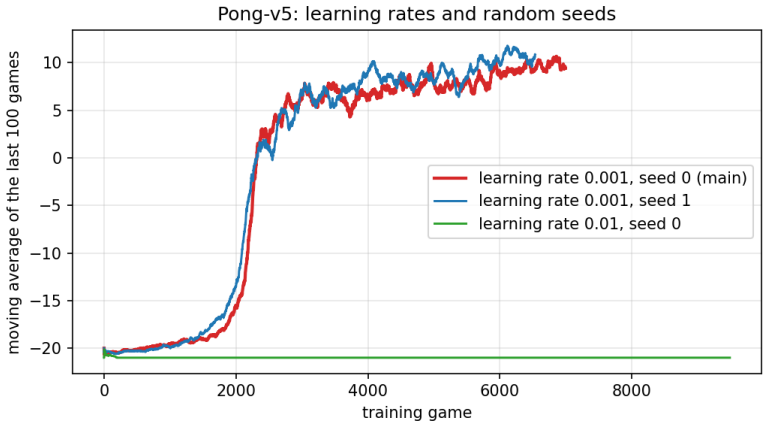


Figure 10. All three runs: learning rate 0.001 with two seeds, and the assignment's learning rate 0.01.

Two seeds, similar results. The second seed at the same settings scored a mean of 10.48 on its 500 test games (standard deviation 4.98), against 10.08 for the main model, and both took off at about the same time. Luck still matters for timing: the earlier, shorter laptop runs with these settings (stopped after about 3,300 games) had ended far apart, at –11.55 and +5.65, so two seeds are still not enough to say how much runs differ.

The assignment's learning rate (0.01) did not work. It stayed at –21 for all 9,486 games (500 test games: mean -21.00). We checked why (test E6): its network quickly became almost always certain of its push, mostly up, so it stopped trying anything else and never found out that other pushes score better. This is why we used 0.001, as the assignment allows when training is not stable.

Pong: Edge cases

The unusual situations we considered for part (b), and how the code handles each one (S1 to S10).

#	Unusual situation	How the code handles it
S1	A lost point blaming pushes from an earlier point	The running total of credit restarts at every point scored, so a push is only credited for its own point.
S2	Bad settings: batch below 1, a learning rate of 0 or below, a discount below 0 or of 1 or more	Rejected at the start with a clear message.
S3	The very first screen of a game, which has no screen before it	The network is shown all zeros for that step instead of failing.
S4	The cleaning function changing the game's own picture	The function edits what it is given, so we always give it a copy.
S5	Using the wrong buttons	Of the 6 actions we use only 2 (up) and 3 (down), and a test confirms what those buttons are called in the game.
S6	A NaN or infinite number in the learning step, or a batch with no games	That update is skipped and counted, so the network cannot be damaged.
S7	Taking the log of a probability that is 0, or huge numbers	Probabilities are built from the raw score, so they stay finite even for extreme networks or inputs.
S8	Running out of time or games in the middle of a batch	The games already played are still learned from instead of being thrown away.
S9	Test games changing the network, or results that cannot be repeated	Test games switch off learning, and random seeds are fixed (the 500 test games use their own seed).
S10	Running out of memory	One game is about 24 MB of inputs, so a batch of 4 is easy to hold.

Pong: Tests

Each test says what we expect to happen, and the result column says what did happen. All 29 tests passed, meaning every result matched the expectation. Run them with `python test_pong.py`. The last column says which edge case (S1 to S10) a test covers.

A. Scoring and game rules

Test (and what we expect)	Result (matched what we expected)	Edge case
A1. Worked by hand: a point won after 3 pushes and one lost after 2 more should give credits 0.81, 0.9, 1, –0.9, –1.	Those were the credits.	S1
A1b. Without the restart, the earlier pushes should be wrongly affected by the later loss (so the restart really does something).	They were: the first credit fell from 0.81 to 0.15.	S1
A2. Bad settings (batch 0, learning rate 0, discount 1.0 or –0.1) should be rejected.	All four were rejected.	S2
A3. It should be ALE/Pong-v5 with 6 actions, where button 2 is up (RIGHT) and button 3 is down (LEFT).	It was.	S5
A4. A full game should end on its own, with rewards only –1, 0 or +1 and a final score between –21 and +21.	A sample game took 944 steps and scored –21.	–
A5. A game should end when one side reaches 21 points.	The sample game ended at 0 points won and 21 lost.	–

B. What the network sees

Test (and what we expect)	Result (matched what we expected)	Edge case
B1. The first screen of a game should give all zeros.	The largest value was 0.	S3
B2. Later inputs should show motion: only –1, 0 and +1, with both the +1 (appeared) and –1 (disappeared) kinds present.	True, with about 25 changed pixels per step.	S3
B3. Cleaning a made-up picture should give 80 x 80 values of 0 or 1, with both background colours erased.	One object pixel was kept.	–
B4. The game's own picture should not be altered by our use of the cleaning function.	It was unchanged.	S4

C. The network

Test (and what we expect)	Result (matched what we expected)	Edge case
C1. Before any training, up and down should be about equally likely.	The average chance of pushing up was 0.503.	–
C2. An extreme network (weights multiplied by 10,000) and extreme inputs (millions) should still give finite log-probabilities.	They did.	S7
C3. One game should fit easily in memory.	It took 24 MB.	S10

D. The training loop

Test (and what we expect)	Result (matched what we expected)	Edge case
D1. A game with a NaN reward should make the update skip, leaving the network untouched.	It was skipped and the weights were identical.	S6
D1b. A NaN inside a game's screen input should make the update skip instead of crashing, leaving the network untouched. (Added after the same test for the baseline code found that the old code crashed here; it was fixed.)	It was skipped and the weights were identical.	S6
D2. An empty list of games, or only empty games, should be skipped instead of crashing.	Both were skipped.	S6
D3. If the limit ends in the middle of a batch (1 game played, batch of 4), that game should still be learned from.	The network changed.	S8
D4. The same seed should give identical results, and a different seed different ones.	Two runs with seed 3 matched exactly; seed 4 differed.	S9
D5. Test games should never change the network.	The weights were identical before and after.	S9
D6. Test games with the same seed should give identical scores.	They did.	S9
D7. A made-up problem with a known answer (one input where pushing up always pays 1): our learning signal should match the exact gradient of the expected reward.	The direction matched (1.0000) and the size was within 1.8%.	–
D7b. The previous check should be able to fail: a learner with the sign flipped, or one that ignores the reward, should be caught.	Both were caught (a direction match of –1.00).	–
D8. In the same made-up problem, training should make the better action much more likely.	Its chance went from 0.50 to 1.00.	–

E. Does it behave as predicted?

Test (and what we expect)	Result (matched what we expected)	Edge case
E1. The saved test result should be exactly 500 games, each scoring between –21 and +21.	500 games, from –7 to 21, a mean of 10.08 and a standard deviation of 4.70.	–
E2. No learning step should have been skipped in the main run.	None was, over 6,993 games.	S6
E3. The trained network should be far better than an untrained one (30 fresh games each, statistical test t above 10).	8.6 against –20.2 (t = 28).	–
E4. Always taking the most likely push should not do much worse than random choice.	11.2 against 8.6.	–
E5. The moving average, worked by hand as in the CartPole tests, should come out right.	All of these matched.	–
E6. The learning-rate-0.01 run should have frozen because its network became almost always certain (above 99% sure) of its push, while the main model stays clearly less certain.	99% of the 0.01 model's pushes were that certain (and 89% were up), against 70% for the main model. Its score was –21.	–

Policy gradient Part 1: With 3 more months

Be more sure of the results. Our runs depend a lot on luck: at the same settings, shorter Pong runs of two seeds ended far apart, and CartPole seeds ranged from 155 to 500. With more time we would run 20 or more seeds for each setting, report averages with confidence ranges, and test whether differences are real or just luck. We would also train Pong for much longer, with a GPU that does not sleep, to see where it levels off.

Test our own choices. We chose to restart the credit at each Pong point, to show the network frame differences, and to learn from 4 games at a time. We would switch each choice off in turn to show that it helps. We would also search properly over learning rate, network size and batch size, and look at why the assignment's learning rate of 0.01 froze on Pong (for example by capping the size of each step, or rewarding some randomness).

Check it a different way. We would try a convolutional network on the Pong pictures, test the trained player against a faster opponent and from different starting points to see whether it only learned one narrow trick, and watch recordings of its games to see how it plays.

Question 3: Policy Gradient, Part 2 (baselines)

What a baseline is. In Part 1, every push was credited with all the reward that followed it. That is noisy: a push can look good just because the whole stretch around it went well, and in CartPole every push is followed by plenty of reward early in a game, even the bad ones. A baseline is a reference number we subtract first, so that *credit = reward that followed – reference*. A push now counts as good only if things went better than the reference, and as bad if they went worse.

Why this helps. The reference must not depend on the push being judged. Then, averaged over many games, learning still points the same way as before, but each learning step is much less noisy. Less noise means steadier learning, and usually faster learning too.

The two baselines we use. A *constant* baseline is one number for every situation: a running average of the rewards from earlier games. A *learned* baseline is a second small network that guesses, for each situation, how much reward to expect from there on, so it can tell an easy moment from a hard one. CartPole uses both; Pong uses the learned one.

Assumptions

These are the choices we made for the baseline experiments on CartPole-v1:

- **Only the baseline changes.** The policy network, discount (0.95), learning rate (0.01), number of games (1500), one update per game, and the random seeds are exactly as in Part 1, so any difference comes from the baseline. “No baseline” in this part is exactly Part 1 (a test checks that the training curves are identical).
- **The constant baseline** is a running average of the rewards from earlier games (about the last 20). Before the first game it is 0. We use earlier games only, because an average that included the current game would depend slightly on the pushes being judged.
- **The learned baseline** is a second network of the same size as the policy (4 inputs, one hidden layer of 128 units, 1 output). It is trained after each game to guess the rewards that were actually earned (squared error), with its own Adam optimiser at the same learning rate. Its guesses are only used as a reference: they carry no learning information into the policy.
- **The same credit as before otherwise:** each push is credited with the reward from that step onwards, discounted by 0.95, minus the reference for that step. Credits are not rescaled.
- **Ten seeds for each method.** A first run with three seeds was too noisy to compare the methods (individual runs at this learning rate can take off or collapse), so we used ten seeds each. The main model (learned baseline, seed 0) was chosen in advance; the other seeds show how much luck matters.
- **Final test:** as in Part 1, 500 new games with random choices and a different random seed.

(a) CartPole-v1 with baselines: graphs and results

The main model is the learned baseline with seed 0, chosen in advance. Figures 11 and 12 show it; Figures 13 and 14 compare all three methods over ten seeds each.

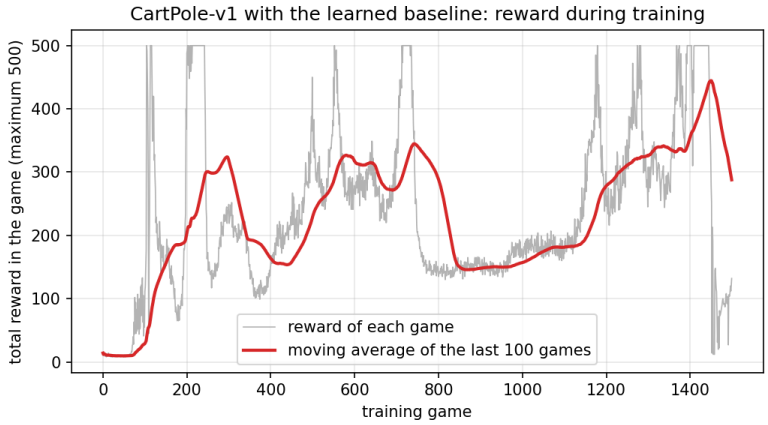


Figure 11. Reward of every training game (grey) and the moving average of the last 100 games (red), main model.

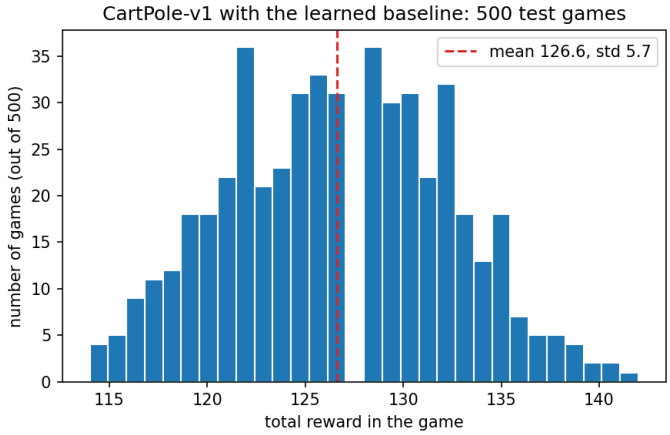


Figure 12. Scores of 500 games played by the trained main model (mean 126.6, standard deviation 5.7).

The main model is not a good advertisement for the method: it scored a mean of 126.6 on the 500 test games, which is below the average of its method. We report it anyway, because we chose it before seeing any results. The comparison over ten seeds is the better guide.

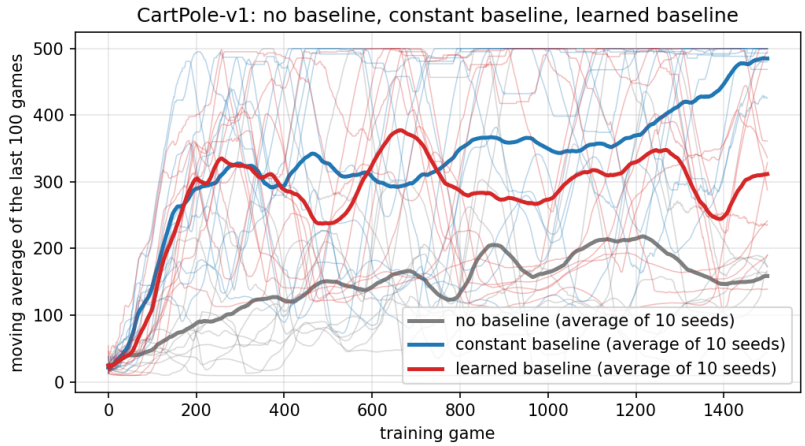


Figure 13. The three methods during training. Faint lines are single seeds, bold lines are the average over ten seeds.

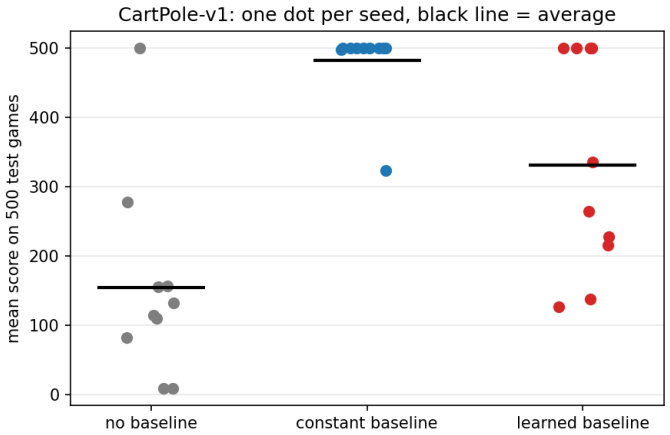


Figure 14. The test score of every seed (500 test games each); the black line is the average of the ten seeds.

Method	Average test score (10 seeds)	Worst and best seed	Seeds at the 500 limit
No baseline	154.9	9 to 500	1 of 10
Constant baseline	482.1	323 to 500	8 of 10
Learned baseline	330.9	127 to 500	4 of 10

Both baselines help. The constant baseline averaged 482 against 155 with no baseline ($p < 0.001$), and the learned baseline averaged 331 ($p = 0.018$). A second statistical test, which makes fewer assumptions, agrees. Only 1 of 10 seeds without a baseline reached the 500 limit, against 8 with the constant baseline and 4 with the learned one. Training is still bumpy with a baseline: the curves in Figure 13 rise and fall.

A surprise. We expected the learned baseline to beat the constant one, because it can tell easy moments from hard ones. It did not: the constant baseline was better ($p = 0.015$). We did not investigate why. One possible reason is that the learned network is trained with a single

noisy step after each game at a high learning rate, so its guesses lag behind. This is a guess, not something we tested, and with more tuning the order could change.

Luck matters a lot, and even tiny differences matter. A first version of this experiment used three seeds and made the learned baseline look worse than no baseline. With ten seeds the picture reversed, so three seeds were not enough. We also found that running the same seed with a different number of computer threads can change a run completely, because tiny rounding differences grow over 1,500 games of training. All results here were made with 1 thread per run, and the tests use the same setting.

CartPole baselines: Edge cases

The unusual situations we considered for the baseline experiments, and how the code handles each one (U1 to U9).

#	Unusual situation	How the code handles it
U1	A baseline that depends on the push it judges (this would bend the direction of learning)	The constant baseline uses earlier games only. The learned baseline is used without learning information, so it cannot be pulled around by the policy's own step.
U2	A NaN or infinite number reaching the baseline network (for example in a game's states)	The update is skipped, and neither network is changed. The baseline's own training step is skipped too if its error is not a usable number.
U3	A NaN or infinite reward	The whole update is skipped, and neither the policy nor the baseline network is changed.
U4	Bad settings, such as an unknown baseline name, 0 games, or a discount of 1 or more	Rejected at the start with a clear message.
U5	An empty game, or a game of only one step	An empty game is skipped. A one-step game still gives a valid update for both baselines.
U6	The new code accidentally changing Part 1	"No baseline" subtracts a reference of exactly 0, and a test checks that it reproduces the Part 1 training curve game for game.
U7	The first games, before the constant baseline has anything to average	It starts at 0, so the first update is the same as in Part 1.
U8	Results that cannot be repeated	Random seeds are fixed, and the 500 test games use a different seed from training.
U9	One lucky or unlucky run deciding the comparison	Ten seeds for each method, and every seed is shown.

CartPole baselines: Tests

Each test says what we expect to happen, and the result column says what did happen. Run them with `python test_cartpole_baselines.py`. The last column says which edge case (U1 to U9) a test covers.

A. The reference and the credits

Test (and what we expect)	Result (matched what we expected)	Edge case
A1. Worked by hand: rewards 3, 2, 1 against a reference of 2 should give credits +1, 0, –1, so the number we minimise is –1.5.	It was –1.50.	–
A2. “No baseline” should be exactly Part 1: the same training curve, game for game.	60 games were compared and all were identical.	U6
A3. In all three settings, the learning step should be exactly the hand-written one (credit = reward – reference).	It was.	U1
A3b. Before the first game the constant baseline is 0, so the first update should be identical to “no baseline”.	It was identical.	U7
A4. The learned baseline should be only a reference for the credits, with no learning information carried into the policy.	Its first use carried none; its second use (its own training) did.	U1
A5. Bad settings (0 games, learning rate 0, discount 1.0, an unknown baseline name) should be rejected.	All were rejected.	U4

B. A baseline should not change where learning points, only how noisy it is

Test (and what we expect)	Result (matched what we expected)	Edge case
B1. Made-up problem with a known answer (one situation where pushing 0 pays 10 and pushing 1 pays 12): with no baseline, a constant baseline and a learned baseline, our learning signal should match the exact gradient of the expected reward.	All three matched: direction match 1.0000 in each, with size errors of 0.092 (none) and 0.001 (constant and learned).	U1
B1b. The previous check should be able to fail: a baseline that peeks at the push’s own reward, or subtracts twice as much, should be caught.	Both were caught (direction match 0.00 and –1.00).	U1
B2. A baseline should make the learning signal much less noisy (repeated small batches of 20 pushes).	The typical spread was 4.95 with no baseline and 1.42 with either baseline.	–
B3. The learned baseline network should learn to expect about the right reward in the made-up problem (about 11).	Its guess was 11.00.	–

C. The training loop

Test (and what we expect)	Result (matched what we expected)	Edge case
C1. A NaN reward should make the update skip, leaving both networks untouched (constant and learned baselines).	It was skipped and all weights were identical.	U3
C1b. A NaN inside a game's states (so the learned baseline outputs NaN) should make the update skip, leaving both networks untouched.	It was skipped and all weights were identical.	U2
C2. An empty game should be skipped instead of crashing (all three settings).	It was skipped.	U5
C3. A one-step game should still give a valid update (constant and learned baselines).	It did, with all numbers finite.	U5
C4. The same seed should give an identical training curve, and a different seed a different one (constant and learned baselines).	Same seed identical, different seed different.	U8

D. Does it behave as predicted? (ten seeds per method)

Test (and what we expect)	Result (matched what we expected)	Edge case
D1. The 500 test scores of every run should be valid (between 1 and 500).	All 30 runs were, from 8 to 500.	–
D2. The learned baseline should do better than no baseline, and the difference should be unlikely to be luck (two statistical tests, both p below 0.05).	330.9 against 154.9, with $p = 0.018$ and 0.008.	U9
D3. The same should hold for the constant baseline.	482.1 against 154.9, with p below 0.001 in both tests.	U9
D4. The saved main model should reproduce exactly the 500 test scores behind the graphs.	All 500 scores were identical (with 1 thread).	U8
D5. Retraining the main model from the same seed should repeat the start of the saved training curve.	The first 200 training games were identical (with 1 thread).	U8

Pong baseline: Edge cases

The unusual situations we considered for the Pong baseline experiment, and how the code handles each one (V1 to V9). Pong uses only the learned baseline: a second network that sees the same screen-difference as the policy.

#	Unusual situation	How the code handles it
V1	A baseline that depends on the push it judges (this would bend the direction of learning)	The baseline network is used without learning information, so the policy's step cannot pull it around. It is trained separately.
V2	A NaN or infinite number in a game's screen input or rewards	The update is skipped before either network is used, and neither is changed. (Writing this test found that the old code crashed instead of skipping; it was fixed, in Part 1 as well.)
V3	The baseline network's own training producing an unusable number	That training step is skipped, and the policy is not affected. (This protection is in the code but is not tested on its own, because an unusable input is already caught earlier; see V2.)
V4	Bad settings: an unknown baseline name (the constant baseline is CartPole only), batch below 1, learning rate of 0 or below, discount of 1 or more	Rejected at the start with a clear message.
V5	An empty game, or a game of only one step	Empty games are skipped. A one-step game still gives a valid update.
V6	The new code accidentally changing Part 1	"No baseline" subtracts a reference of exactly 0, and a test checks that one learning step gives identical weights to the Part 1 code.
V7	Credit leaking across points	As in Part 1, the credit restarts at each point, and the baseline is trained to guess the same restarted credit.
V8	Running out of time or games in the middle of a batch	The games already played are still learned from.
V9	Results that cannot be repeated	Random seeds are fixed, and the 500 test games use their own seed.

Pong baseline: Tests

Each test says what we expect to happen, and the result column says what did happen. Run them with `python test_pong_baseline.py`. The last column says which edge case (V1 to V9) a test covers.

A. The reference and the credits

Test (and what we expect)	Result (matched what we expected)	Edge case
A1. The learning step should be exactly the hand-written one: credit = (reward that followed, restarting at each point) − reference, averaged over the games, with and without the learned baseline.	It was.	V1, V7
A2. "No baseline" should be exactly Part 1: one learning step should give identical weights.	The weights were identical.	V6
A3. The learned baseline should be only a reference for the credits, with no learning information carried into the policy.	Its first use carried none; its second use (its own training) did.	V1
A4. Bad settings (batch 0, learning rate 0, discount 1.0, an unknown baseline name) should be rejected.	All four were rejected.	V4

B. A baseline should not change where learning points, only how noisy it is

Test (and what we expect)	Result (matched what we expected)	Edge case
B1. The learned baseline network should learn to expect about the right reward in a made-up problem (pushing down pays 10, pushing up pays 12, so about 11).	Its guess was 11.11.	–
B2. Made-up problem with a known answer: with no baseline and with a learned baseline, our learning signal should match the exact gradient of the expected reward.	Both matched: direction match 1.0000 in each, with size errors of 0.066 (none) and 0.001 (learned).	V1
B2b. The previous check should be able to fail: a baseline that peeks at the push's own reward, or subtracts twice as much, should be caught.	Both were caught (direction match 0.00 and −1.00).	V1
B3. A baseline should make the learning signal much less noisy (repeated small batches of 20 pushes).	The typical spread was 8.44 with no baseline and 3.18 with the learned baseline.	–

C. The training loop

Test (and what we expect)	Result (matched what we expected)	Edge case
C1. A NaN reward should make the update skip, leaving both networks untouched.	It was skipped and all weights were identical.	V2
C1b. A NaN inside a game's screen input should make the update skip instead of crashing, leaving both networks untouched.	It was skipped and all weights were identical (after the fix described under V2).	V2
C2. An empty list of games, or only empty games, should be skipped instead of crashing (both settings).	They were skipped.	V5
C3. A one-step game should still give a valid update.	It did, with all numbers finite.	V5
C4. The same seed should give an identical training curve, and a different seed a different one (real games).	Same seed identical, different seed different.	V9
C5. If the game limit ends in the middle of a batch (1 game played, batch of 4), that game should still be learned from.	The network changed.	V8